SASS

Installation

1. sassmeister.com
2. sass is written in ruby
3. Local installation
    1. Mac OSX, Linux
        1. `gem -v`
        2. `sudo gem install sass`
        3. `sass -v`
    2. Windows:
        1. Download Ruby 1.9.3
        2. Double click to install
        3. Open cmd in admin mode
            1. `gem install sass`
            2. `sass -v`
4. Prerequisite:
    1. `sudo apt-get install ruby -y`

SASS Commands

1. `sass -h` **(M)**

    ```
    cd desktop
    sass-convert Example.scss Example.sass
    sass-convert Example.sass Example.scss
    ```

2. Convert scss to css

    ```
    sass Example.scss:Example.css
    ```

    1. Exmaple.map: used for debugging

3. Convert dynamically

    ```
    sass --watch Example.scss:Example.css
    ```

SASS Commands

1. `sass -h` : options, inputs and outputs
2. `cd Desktop/SASS_Resources`
3. Converting SCSS to SASS file:
    1. `sass-convert Example.scss Example.sass` **(M)**
4. Reversing:
    1. `sass-convert Example.sass Example.scss`
5. Converting SCSS to CSS file:
    1. `sass Example.scss:Example.css` **(M)**
        1. Generates a map file as well (used for debugging)
6. Automatically convert to CSS
    1. `sass --watch Example.scss:Example.css` **(M)**
        1. When we change SCSS file, CSS file automatically changes
7. Automatically convert CSS in a directory:
    1. `sass --watch Directory:CSS`
        1. Generates `CSS` directory
8. Remove `CSS`
9. `sass Example.scss:Example.css`
10. Choosing format:
    1. `sass Examle.scss:Example.css --style compact`
    2. `sass Example.scss:Example.css --style compressed`
    3. `sass Example.scss:Example.css --style expanded`
11. Map:
    1. `sass Example.scss:Example.css --style expanded --sourcemap=none`
        1. Map file is not generated
12. Watching:
    1. `sass --watch Example:scss:Example.css --style expanded --sourcemap=none`
    2. `sass --watch Directory:CSS --style compact --sourcemap=none`

Environment Setup

1. Editor: Atom
2. Splitting pane:
    1. View > Panes > Split Right
3. Splitting further:
    1. Select Left side pane
    2. View > Panes > Split Down

4. New file:
    1. Select left side top pane
    2. New file > SCSS.scss
    3. Disable Tree view
    4. Select bottom left pane
    5. New file > SASS.sass
    6. Select right side pane
    7. New file > CSS.css
5. `cd Desktop`
    1. Watch and automatically convert
        1. `sass --watch SCSS.scss:CSS.css --style expanded --sourcemap=none`
    2. convert to SASS
        1. `sass --convert SCSS.scss SASS.sass`
6. There is only syntactical difference between SCSS and SASS
    1. SASS ommits `;`
    2. SASS omits `{` and `}`
        1. Indents instead

Basic API

Variables

1. Example: Using variables instead of color value
2. Syntax: SCSS

    ```
    $<variable-name>: <value>;
    ```

3. Syntax: SASS

    ```
    $<variable-name>: <value>
    ```

4. Data types:
    1. numerical: 600px, 600em
    2. Floating point value: 1.5
    3. Integer: 20
5. Example:

```
$width: 600px;
$int: 20;

div {
    width: $width;
    line-height: $int;
}
```

6. Strings:
    1. ""
    2. "
    3.
    4. Example:

```
$string: "../images/hello.jpg"

background: url($string) no-repeat red;
```

7. CSS Selector:

```
div .#{ $stringSelector } {
    background: ...
}
```

8. SASS converts rgb(...) to hex
9. List data type:
    1. List is a sequence separated by spaces or commas
    2. Example: `$list: 1px 1px 3px black, 3px 3px 4px #ccc;`
10. Maps:
11. bool
12. null: `$null` : null - no value

Inheritance

1. Types:

1. Selector inheritance

```
$el: button;

#parent {
    background-color: green;

    #child-One, .another, .evenmore {
        background: yellow;
        font-size: 1em;
    }

    #child-Two {
        font-weight: 200;

        #inner-child .content {
            background: blue;
        }
    }
}

&.HasClass {
    background-color: blue;
    font-weight: bold;
    font-style: italic
}
```

2. Property inheritance

```
%shared {
    background: green;
    font-size: 2em;
}

.content { @extend %shared; }
```

    1. %: marks a place holder and no CSS is produced (This can be shared across CSS elements)
    2.

```
#parent %shared .child {
    background: green;
    font-size: 2em;
}
```

```
.content { @extend %shared; }
```

3. Other properties:

```
.content {
    @extend %shared;
    line-height: 2em;
}

.sign, video, img {
    @extend %shared;
}
```

    1. Repeats for all the above elements

4. For list

```
$list: elementOne, elementTwo, elementThree;

.sign, video, img, #{ $list } {
    @extend %shared;
}
```

Mixins

1. Mixins are passed to SASS before the file is compiled to CSS
    1. Can be SASS or CSS or both
    2. Can take arguments to convert the output
2. Example:

```
@mixin cssProperties {
    color: blue;
    background: green;
    line-height: 1.8;
}

body {
    @include cssProperties();
}
```

3. Example:

```
@mixin cssProperties {
    #main {
        color: blue;
        background: green;
        line-height: 1.8;
    }
}

body {
    color: green;
    width: 200px;

    @include cssProperties();
}
```

4. Arguments:

```
@mixin cssProperties($argOne) {
    #main {
        color: $argOne;
        background: green;
        line-height: 1.8;
    }
}
```

    1. Must call it will value

```
body {
    @include cssProperties(green);
}
```

    2. We can pass another variable

    3. Multiple arguments:

```
@include cssProperties($color, 2.6);
```

    4. Example:

```
@mixin boxShadow($shadow) {
    -webkit-box-shadow: $shadow;
    -moz-box-shadow: $shadow;
    -o-box-shadow: $shadow;
    box-shadow: $shadow;
}

body {
    @include boxShadow(1px 2px 3px rgba(0, 0, 0, .3));
}
```

5. List of data

```
@mixin boxShadow($shadow...) {
    box-shadow: $shadow;
}

body {
    @include boxShadow(1px 2px 3px rgba(0, 0, 0, .3), 2px 4px 3px rgba(0, 0, 0, .24));
}
```

5. `@content` directive

1. Example:

```
@mixin keyframes($name) {
    @-webkit-keyframes #{ $name } {
        @content;
    }

    @-moz-keyframes #{ $name } {
        @content;
    }

    @-o-keyframes #{ $name } {
        @content;
    }

    @keyframes #{ $name } {
        @content;
    }
```

```
}

@include keyframes(myAnim) {
    0% { opacity: 0 }
    100% { opacity: 1 }
};
```

1. content is inserted in place of `@content`
2. SASS:
    1. No `@mixin` but prefixed with `=`

Import

1. We can build our own library and share
2. Syntax:

```
@import 'import.css';
```

3. Syntax: SCSS file

```
@import '_MainMixins'; /* file extension is not required for SCSS and SASS */

html {
    background: $SCSSVariable;
    @include boxShadow(1px 2px 3px rgba(0, 0, 0, .4), 1px 2px 3px red);
    @include transition(bancground .2s, height .4s);
}

@include keyframes(myAnim) {
    0% { opacity: 0 }
    100% { opacity: 1 }
}
```

4. Import SASS file:

```
@import '_import.sass';

html {
```

```
        @include properties(green);
    }
```

1. We can import SASS inside SCSS and vice versa
2. _ : tells they are partials which are used in other files
    1. Don't get exported as css files

5. Convertion:
    1. `sass --watch SASS_Import:CSS --style expanded --sourcemap=none`
    2. Does not copy .css files

Advanced API

List Functions

1. A list with a one value is also a list

```
$List1: 200px;
$List2: 1px 2px 3px black,;
$List3: Helvetica, Arial, sans-serif;

$NestedList: 1px 2px 3px green, 6px 7px 8px blue, 3px 2px 1px yellow;

html {
  background: nth($NestedList, 2);
  background: set-nth($NestedList, 2, 2px 3px 5px red);
  background: list-separator($NestedList);
  background: join($List2, $NestedList, comma);
  background: join($List2, 'opensans', space);
  background: append($List2, 'opensans', space);
  background: index($List3, Arial);
  background: zip(
    1px 2px 3px 4px,
    4em 3em 2em 1em,
    black green blue red
  )
}

// $BasicMap:(=);

// $Map1: (=
```

```
// );


html {
  background: 6px 7px 8px blue;
  background: 1px 2px 3px green, 2px 3px 5px red, 3px 2px 1px yellow;
  background: comma;
  background: 1px 2px 3px black, 1px 2px 3px green, 6px 7px 8px blue, 3px
  background: 1px 2px 3px black "opensans";
  background: 1px 2px 3px black "opensans";
  background: 2;
  background: 1px 4em black, 2px 3em green, 3px 2em blue, 4px 1em red;
}
```

Maps

1. To build hierarchy: Nesting an object within an object

```
$map1: (
  'KeyName': 'Value',
  'AnotherKey': 200px
);

$map2: ('key1': 'value1', 'key2': 'value2', 'key3': 'value3');

$map3: (
  'KeyName': 'targeting a key produces its value!',
  'String': string,
  'Number': 102,
  'List': (background .2s, color .2s),
  'NestedMap': (
    'NestedKey': 'Nested value',
    'url': './images/nestedimg.png',
    'Width': 300px
  )
);

.foo {
      content: map-keys($map1);
      content: map-values($map1);
      content: map-has-key($map1, 'KeyName');
      content: map-get($map1, 'KeyName');
```

```
        content: inspect(map-merge($map1, $map2)); // represented as a stri
        content: inspect(map-remove($map2, 'key1', 'key2'));
}


.foo {
  content: "KeyName", "AnotherKey";
  content: "Value", 200px;
  content: true;
  content: "Value";
  content: ("KeyName": "Value", "AnotherKey": 200px, "key1": "value1", "k
  content: ("key3": "value3");
}
```

Math Operators & Functions

1. Addition, multiplication, etc.

```
$a: 10px;
$b: 32px;
$c: 4.5px;

$eq: 4 - (6 + $b);

div {
  width: 5 + 10 + $a - $b * 2 / $c;
  width: round(5 + 10 + $a - $b * 2 / $c);
  width: floor(5 + 10 + $a - $b * 2 / $c);
  width: ceil(5 + 10 + $a - $b * 2 / $c);
  width: percentage(1.1); // No CSS measurement allowed
  width: abs(-12px);
  width: min(10, 20, 30, 40, 50);
  width: max(10, 20, 30, 40, 50);
  width: random();
  width: random(100); // inclusive of the limit
  height: $b % $c;
  line-height: 4 - (6 + 5);
  line-height: $eq;
}


div {
  width: 10.7777777778px;
```

```
  width: 11px;
  width: 10px;
  width: 11px;
  width: 110%;
  width: 12px;
  width: 10;
  width: 50;
  width: 0.9331597896;
  width: 2;
  height: 0.5px;
  line-height: -7;
  line-height: -34px;
}
```

If Directives

```
$type: element;
$num: 10;
$noValue: null;
$eq: $num + 1;

html {
  @if $type == element {
    content: 'Yup TRUE!';
  } @else if $type != helloWorld {
    content: 'Doesn\'t match helloWorld';
  } @else if $type != null {
    content: 'Doesn\'t match null';
  } @else {
    content: 'Nope FALSE!';
  }

  @if $num == 10 {
    width: 10px;
  } @else if $num == 11 {
    width: 0px;
  }

  @if $num + 1 == 11 {
    width: 11px;
  }
}
```

```
@if $eq == 11 {
  body {
    width: 11px;
  }
}


html {
  content: "Yup TRUE!";
  width: 10px;
  width: 11px;
}

body {
  width: 11px;
}
```

Loops

```
$headers: h1, h2, h3, h4, h5, h6;
$fontSize: 55px, 45px, 40px, 35px, 30px, 25px;

$index: 1;

@for $i from 1 through 6 {
  h#{$i} {
    font-size: 55px;
  }
}

@for $i from 6 through 1 {
  h#{$i} {
    font-size: 55px;
  }
}

$len: length($headers);

@for $i from 1 through $len {
  #{nth($headers, $i)} {
    font-size: nth($fontSize, $i);
    line-height: nth($fontSize, $i) + 10;
```

```
  }
}

@each $head in h1, h2, h3, h4, h5, h6 {
  #{$head} {
    font-size: 55px;
  }
}

@each $head in $headers {
  #{$head} {
    font-size: 55px;
  }
}

@each $head, $font, $color in (h1, 55px, green), (h2, 50px, blue) {
  #{$head} {
    font-size: $font;
    color: $color;
  }
}

$headersMap: (
  h1: 55px,
  h2: 50px,
  h3: 45px
);
@each $head, $font in $headersMap {
  #{$head} {
    font-size: $font;
  }
}

$index: 1;
@while $index <= 6 {
  h#{$index} {
    font-size: 55px;
  }
  $index: $index + 1;
}

$index: 1;
@while $index <= 6 {
  #{nth($headers, $index)} {
```

```
    font-size: nth($fontSize, $index);
  }
  $index: $index + 1;
}


h1 {
  font-size: 55px;
}

h2 {
  font-size: 55px;
}

h3 {
  font-size: 55px;
}

h4 {
  font-size: 55px;
}

h5 {
  font-size: 55px;
}

h6 {
  font-size: 55px;
}

h6 {
  font-size: 55px;
}

h5 {
  font-size: 55px;
}

h4 {
  font-size: 55px;
}

h3 {
  font-size: 55px;
}
```

```
h2 {
  font-size: 55px;
}

h1 {
  font-size: 55px;
}

h1 {
  font-size: 55px;
  line-height: 65px;
}

h2 {
  font-size: 45px;
  line-height: 55px;
}

h3 {
  font-size: 40px;
  line-height: 50px;
}

h4 {
  font-size: 35px;
  line-height: 45px;
}

h5 {
  font-size: 30px;
  line-height: 40px;
}

h6 {
  font-size: 25px;
  line-height: 35px;
}

h1 {
  font-size: 55px;
}

h2 {
```

```
  font-size: 55px;
}

h3 {
  font-size: 55px;
}

h4 {
  font-size: 55px;
}

h5 {
  font-size: 55px;
}

h6 {
  font-size: 55px;
}

h1 {
  font-size: 55px;
}

h2 {
  font-size: 55px;
}

h3 {
  font-size: 55px;
}

h4 {
  font-size: 55px;
}

h5 {
  font-size: 55px;
}

h6 {
  font-size: 55px;
}

h1 {
```

```
  font-size: 55px;
  color: green;
}

h2 {
  font-size: 50px;
  color: blue;
}

h1 {
  font-size: 55px;
}

h2 {
  font-size: 50px;
}

h3 {
  font-size: 45px;
}

h1 {
  font-size: 55px;
}

h2 {
  font-size: 55px;
}

h3 {
  font-size: 55px;
}

h4 {
  font-size: 55px;
}

h5 {
  font-size: 55px;
}

h6 {
  font-size: 55px;
}
```

```
h1 {
  font-size: 55px;
}

h2 {
  font-size: 45px;
}

h3 {
  font-size: 40px;
}

h4 {
  font-size: 35px;
}

h5 {
  font-size: 30px;
}

h6 {
  font-size: 25px;
}
```

Function Directives

```
$headers: h1, h2, h3, h4, h5, h6;
$fontSize: 55px, 45px, 40px, 35px, 30px, 25px;

@function list-match($list, $list2) {
  $len: length($list); // in function scope only
  $len2: length($list2);

  @if $len == $len2 {
    @return "Yup"
  } @else {
    @return "Nope"
  }
}

html {
```

```
  width: list-match($headers, $fontSize);
}


html {
  width: "Yup";
}
```

SASS extended

SASS Extended Introduction & Koala Compiler

1. Koala app
    1. GUI application for Less, Sass, Compass, and CoffeScript
        1. https://koala-app.com/
    2. Drag and drop
        1. Open output folder
    3. Outputs:
        1. Source map
        2. Auto Compile (auto-watch)
        3. Debug Info
        4. Line comments
        5. Unix newlines
        6. Autoprefix (?)
        7. Output style
            1. nested
            2. expanded (dev)
            3. compact
            4. compressed (production)

SASS compiler auto prefix css properties

1. Autoprefix
    1. Sees any CSS property that needs prefixing and automatically applies it (no mixin required)

```
a {
  transition: all .2s;
}
a {
  -webkit-transition: all .2s;
```

```
            transition: all .2s;
    }
```

SASS Source Maps

1. Source-map
    1. Mapping out original source (SCSS file)
        1. SCSS code will show up in the developer tools in the browser
            1. Map will map source code into dev tools
2. Example: index.html

```
<html>
  <head>
    <title>Sass Extended</title>
    <link rel="stylesheet" href="Example.css"/>
  </head>
  <body>
    <h1>Sass Extended</h1>
  </body>
</html>
```

```
$gray: #aaa;

@mixin txtStyle($size, $weight, $lh) {
  padding: 0;
  margin: 0;
  font-size: $size;
  font-weight: $weight;
  line-height: $lh;
}

h1 {
  color: $gray;
  @include txtStyle(25px, 200px, 140%);
}
```

```
h1 {
  color: #aaa;
  padding: 0;
  margin: 0;
```

```
  font-size: 25px;
  font-weight: 200px;
  line-height: 140%; }

/*# sourceMappingURL=Example.css.map */
```

3. Select: Source Map (Example.css.map)
4. Enable CSS source maps in browser
    1. Auto-relload generated CSS
5. Click on `Example.scss`
    1. Allows us to make changes in Dev tools, which are automatically saved to file on disk
        1. Right click on Example.scss
        2. Add Folder to Workspace
        3. Select `SASS` workspace
        4. Allow access
        5. Open SASS/Example.scss
            1. We can update and save
    2. We can do it with any file
        1. `index.html`
            1. It reflects in a file
6. Firefox, Safari
    1. SCSS is mapped

SASS Grid

1. Grid.scss

```
#grid > div {
  height: 25px;
  background: #ccc;
  border: 1px solid #aaa;
  box-sizing: border-box;
  float: left;
}

$max: 16;

@for $i from 1 through $max {
  $equ: 100% / $i;

  #grid.cell-#{$i} > div {
    width: $equ;
```

```
  }
}

#grid > div {
  height: 25px;
  background: #ccc;
  border: 1px solid #aaa;
  -webkit-box-sizing: border-box;
          box-sizing: border-box;
  float: left; }

#grid.cell-1 > div {
  width: 100%; }

#grid.cell-2 > div {
  width: 50%; }

#grid.cell-3 > div {
  width: 33.3333333333%; }

#grid.cell-4 > div {
  width: 25%; }

#grid.cell-5 > div {
  width: 20%; }

#grid.cell-6 > div {
  width: 16.6666666667%; }

#grid.cell-7 > div {
  width: 14.2857142857%; }

#grid.cell-8 > div {
  width: 12.5%; }

#grid.cell-9 > div {
  width: 11.1111111111%; }

#grid.cell-10 > div {
  width: 10%; }

#grid.cell-11 > div {
  width: 9.0909090909%; }
```

```
#grid.cell-12 > div {
  width: 8.3333333333%; }

#grid.cell-13 > div {
  width: 7.6923076923%; }

#grid.cell-14 > div {
  width: 7.1428571429%; }

#grid.cell-15 > div {
  width: 6.6666666667%; }

#grid.cell-16 > div {
  width: 6.25%; }

/*# sourceMappingURL=Grid.css.map */
```

SASS Grid Media Queries

1. Grid.scss

```
#grid > div {
  height: 25px;
  background: #ccc;
  border: 1px solid #aaa;
  box-sizing: border-box;
  float: left;
}

$max: 16;

@for $i from 1 through $max {
  $equ: 100% / $i;

  #grid.cell-#{$i} > div {
    width: $equ;
  }
}

$map: (
  850: 12,
  450: 6,
  380: 3,
```

```
);

@each $breakpoint, $cols in $map {
  @media all and (max-width: #{$breakpoint}px) {
    @for $i from 1 through $cols {
      $equ: 100% / $i;

      #grid.cell-#{$breakpoint}-#{$i} > div {
        width: $equ;
      }
    }
  }
}


<html>

<head>
   <title>Sass Grid</title>
   <link rel="stylesheet" href="Grid.css" />
</head>

<body>
   <div id="grid" class="cell-16 cell-850-12 cell-450-6 cell-380-3">
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
      <div></div>
   </div>
</body>

</html>
```

SASS Buttons

```
.button {
  display: inline-block;
  text-decoration: none;
  font-family: "helvetica neue", helvetica, arial, sans-serif;
  font-size: 14px;
  font-weight: #ccc;
  background: #ccc;
  color: #222;
  padding: 10px 15px;
  border-radius: 3px;
  transition: all 0.3s ease-in-out;
}

.button:hover {
  background: #bbb;
  color: #000;
}

$colormap: (
  green: (
    background: #4caf50,
    hover: #45a049,
  ),
  blue: (
    background: #008cba,
    hover: #007bb5,
  ),
  red: (
    background: #f44336,
    hover: #da190b,
  ),
);

@each $color, $map in $colormap {
  @each $prop, $val in $map {
    @if ($prop == "background") {
      .button.#{$color} {
        background: $val;
        color: #fff;
      }
    } @else if ($prop == "hover") {
```

```
      .button.#{$color}:hover {
        background: $val;
      }
    }
  }
}


/* line 1, ../../sass/06+Extended/Buttons.scss */
.button {
  display: inline-block;
  text-decoration: none;
  font-family: "helvetica neue", helvetica, arial, sans-serif;
  font-size: 14px;
  font-weight: #ccc;
  background: #ccc;
  color: #222;
  padding: 10px 15px;
  border-radius: 3px;
  -webkit-transition: all 0.3s ease-in-out;
  transition: all 0.3s ease-in-out; }

/* line 14, ../../sass/06+Extended/Buttons.scss */
.button:hover {
  background: #bbb;
  color: #000; }

/* line 37, ../../sass/06+Extended/Buttons.scss */
.button.green {
  background: #4caf50;
  color: #fff; }

/* line 42, ../../sass/06+Extended/Buttons.scss */
.button.green:hover {
  background: #45a049; }

/* line 37, ../../sass/06+Extended/Buttons.scss */
.button.blue {
  background: #008cba;
  color: #fff; }

/* line 42, ../../sass/06+Extended/Buttons.scss */
.button.blue:hover {
  background: #007bb5; }
```

```
/* line 37, ../../sass/06+Extended/Buttons.scss */
.button.red {
  background: #f44336;
  color: #fff; }

/* line 42, ../../sass/06+Extended/Buttons.scss */
.button.red:hover {
  background: #da190b; }

/*# sourceMappingURL=Buttons.css.map */
```

2. Use cases: Variations with repeated code
    1. Color schemes

Bonus Lecture: Discount courses

1. Code: avelx5050